
Assignment 10**Learning goals:**

Apply the concepts and methods presented in chapter "Code Generation" of the lecture to some exercise problems.

Presentation:

Please prepare the solutions for informal presentation in class, and fill in the online check list in ZEUS.

Task 1 If statements and conditions

Determine the sequence of MJVM instructions generated by the attributed grammar presented on lecture slides 70-74 for the following if statement. For each generated instruction, give the production and semantic action of the attributed grammar that emits the instruction. For jump instructions, also explain when the target address is written to the code buffer. x and y are global variables at addresses 0 and 1, respectively.

```
if (x < 0 || x > 9 && x < 100) y = 1;
else y = 0;
```

Task 2 Switch statements – ATG

Write a production *SwitchStatement* of an attributed grammar for translating a switch statement to MicroJava byte code. An example of a switch statement is given below (variable declarations are not shown). The switch expression should be restricted to the integer data type, and case labels should be numbers (constants). Cases need to contain *break* statements unless control flow should "fall through" to the next case - this is the same semantics as for well-known programming languages like Java or C. Note that the MicroJava compiler already supports break statements - no modification needed (see lecture slide 72).

```
switch (x % 5) {
  case 0: y = x / 5; n0++; break;
  case 1: y = (x-1) / 5; n1++; break;
  case 2: n2++; // fall through
  default: y = x;
}
```

Task 3 Switch statements – implementation

Extend the MicroJava compiler to support switch statements as defined in Task 2. Document the changes to the MJ compiler source code and test your compiler with multiple input programs (with and without switch statements). Make sure that compiled programs can be executed by the MJVM and deliver the expected output.

Task 4 Method declaration and calls

Compile the following source program using the provided (unmodified) MJ compiler and decode the generated object file using MJ.Decode. Locate and explain the MJVM instructions `enter`, `exit`, `return`, and `call` (including operand values) in the decoded output. Then execute the object file using MJ.Run with `-debug` option, and explain what happens on method and expression stack when method `sum2` is called and when it returns to the caller.

```
program MethodCall
{
    int sum2(int min, int max)
        int i, s;
        {
            s = 0;
            i = min;
            while (i <= max) {
                s = s + i*i;
                i++;
            }
            return s;
        }

    void main()
    {
        print(sum2(2,4));
    }
}
```